

Java

¿Qué es Java? → es un lenguaje orientado a objetos, pero es multiparadigma, y multiplataforma.

- desarrollado por sun Microsystems y ahora adquirido por Oracle (quien lo distribuye gratuitamente)
- cada versión la saca en 3 productos
 - JDK (standard edition) → este vamos a usar
 - EE (Enterprise edition) → paginas web
 - ME (Micro edition) → smartphone
- basado en C y C++, pero sin punteros (lenguaje simple).
- Características
 - robusto
 - simple
 - O.O

¿Por qué Java?

- objetivo inicial: crear un lenguaje multiplataforma
- Write once (escribo una vez y puedo ejecutar en cualquier sistema operativo)

Características

- Seguro: no puedo codificar en java un programa que acceda al disco de la máquina remoto.
- Tipado estáticamente: todas las cosas que vamos a definir tenemos que decir que tipo es.
- Portable: me lo llevo de un dispositivo a otro dispositivo. (no importa el sistema operativo) → independiente de la arquitectura
- MultiThread (múltiples ciclos): **NO ENTENDI**
- Interpretado Mixto: es interpretado pero hace una compilación (just in time) previa que no llega a binario(me ayuda a saber si tengo error de sintaxis), entonces decimos que interpreta una versión pseudo compilado.
- Dinámico: ante un cambio automáticamente recompila lo que detecta como modificación.

Interpretado (Como pasamos de “print” a 0 y 1): se traduce a binario línea a línea a medida que voy ejecutando (tarda más en ejecutar).
Complicado: traduce todo a binario antes de ejecutar.

Para qué sirve Java → páginas web, smart tv, smart phone, etc..
(objetivo general)

Desarrollo Java

- Archivos fuente: el nombre de la clase que definen .java
- Compilación:
 - ¿Cómo se compila? javac nombreClase.java
 - genera el pseudocódigo bytecode, con la extensión “.class” este es el archivo que tengo que llevar de máquina en máquina para que funcione el programa.
- Intérprete: “Java Virtual Machine”, permite la ejecución.
- Ejecución:
 - ¿Cómo se ejecuta? java nombreClase
- Ventajas:
 - NO reescribir código.
 - compatible con cualquier máquina.
- Desventajas:
 - Velocidad
 - Difícil

Pasos:

HolaMundo.java → javac HolaMundo.java → HolaMundo.class → java HolaMundo

→ Versiones de Java:

- JRE → el entorno mínimo para ejecutar programas Java. Incluye la JVM y la API. Está incluida en el JAVA SE
- Hay versiones que son LTS (Long Term Support) → versiones con soporte prolongado, (cada 3 o 4 años)
- Cualquier versión es compatible con las versiones anteriores.

ATENCIÓN: Java solo interactúa con la JVM. Independencia del SO y HW

Identificadores:

- Nombran variables, funciones, clases y objetos.
- **Comienzan con cualquier letra, un subrayado (_) o el símbolo de pesos (\$).**
- Distinguen las mayúsculas de las minúsculas.
- No poseen longitud máxima.
- Convenciones:
 - Clases: sustantivos en singular.
 - Dato miembro: Sustantivo singular
 - Método miembro: Verbo en infinitivo o verbo más sustantivo
 - Constantes: Todo con mayúscula y separado de palabras con “_” (constantes, cuando la declaramos le asignamos el valor).

Variables:

- Variables de Objetos: cada uno tiene una posición de memoria , definidas dentro de un objeto, no son estáticas (una factura)

- Variables de Clases: valor compartido para todos los objetos de la clase, estáticas, públicas y el valor puede modificarse (mismo nombre en muchas facturas)
- Variables Locales: sólo son visibles dentro del método que fueron definidas.
- Parámetros:
- Variables Globales: NO TIENE

→ las variables(minúscula) locales tienen que ser inicializadas, los tipos de datos(minúscula):

Visibilidad:
 public: visibles desde cualquier lugar
 private: visible solo desde la misma clase
 protected: visible desde dentro de la misma clase, subclases y clases del mismo paquete

- **Dos tipos de variables:**
 - **Tipos primitivos de datos:**

Tipos Datos	Tamaño (bits)	Valor inicial
byte	8	0
short	16	0
int	32	0
long	64	0
float	32	0.0F
double	64	0.0D
char	16	\0
boolean	8	false

- **byte** i;
 - **short** depto, piso;
 - **int** edad, cant = 100;
 - long suficiente;
 - **float** precio= 10.6f, promedio =3.25f;
- agregando la f me permite castear, le aviso al compilador que lo trate como un flotante y no como un double
- **double** peso = 15450.834;
 - **char** letra = 'a'; → son expresados como un entero

ATENCIÓN: String no es un tipo de dato primitivo

IMPORTANTE: Los casteos los hago solo cuando pierdo precisión.

- **Variables de referencia:**
 - **Vectores**
 - se pueden declarar vectores o array de cualquier tipo primitivo, incluso array de array (matrices)
 - son objetos que tienen métodos asociados
 - se dimensionan en tiempo de ejecución

Operadores de Asignación:

Operadores Unarios:

Operadores de comparación:

Operadores Lógicos:

- & AND (completo)
- && AND (cortocircuito)
- | OR (completo)
- || OR (cortocircuito)
- ^ XOR
- ! NOT (unario)

Operadores de bit:

- & AND
- | OR
- ^ XOR
- << Desplazamiento a la izquierda
- >> Desplazamiento a la derecha
- >>> Llenado de ceros a la derecha
- ~ Complemento de bits

Operadores de bits y asignación:

- &= Asignación AND ($x = x \& y$)
- |= Asignación OR ($x = x | y$)
- ^= Asignación XOR ($x = x ^ y$)
- <<= Asignación del desplazamiento a la izquierda ($x = x << y$)
- >>= Asignación del desplazamiento a la derecha ($x = x >> y$)
- >>>= Asignación de llenado de ceros hacia la derecha ($x = x >>> y$)

Precedencia:

1. Unarios: +, -, ++, --, !, ~
2. Multiplicación, división: *, /, %
3. Suma y resta: +, -
4. Desplazamiento: <<, >>, >>>
5. Relacionales: <, >, <=, >=
6. Igualdad: ==, !=
7. Lógicos completos y cortocircuito: &, ^, |, &&, ||
8. Condicionales: ? : (abreviación de if/then/else)
9. Asignación: =, +=, -=, *=, /=, %=, &=, |=, ^=, <<=, >>=, >>>=

Caracteres especiales

- () paréntesis:
 - Listas de parámetros en la definición y llamada a métodos
 - Precedencia en expresiones
 - Expresiones booleanas
- Conversión de tipos {} llaves:
 - Bloque de sentencias de código (clases, métodos)
 - Valores de matrices inicializadas automáticamente
- [] corchetes:
 - Declaración y referencia a valores de vectores y matrices
- ; punto y coma:
 - Separador de sentencias
- , coma:
 - Separador de: declaraciones de variables, lista de argumentos, elementos de arrays, incremento/decremento en una sola línea
- . punto:
 - Referencia a un método o propiedad de un objeto o de una clase (operador de ámbito).

Técnicas para el control de flujo en JAVA

1. Salto condicional

→ IF:

- puede usar {} o ;

→ Switch: se usa con valores FIJOS (muchos 6+)

- las sentencias van sin llave
- NO VALE PARA CUALQUIER TIPO DE DATO
- puede ser más de una sentencia
- default: se ejecuta solo cuando no se cumpla ninguno de los casos anteriores

```
switch (expresion) {
    case valor1:
    case valor2:
        sentencias;
        break;
    case valor3:
        sentencias;
        break;
    default:
        sentencias;
        break;
}
```

- **expresion:** solo acepta los siguientes tipos de datos:
 - byte
 - char
 - short
 - int
 - String: a partir de la versión 7.0
- **break:** evita la ejecución de las sentencias indicadas en otros valores consecutivos.

→ TRY-CATCH

- Se genera en la JVM como consecuencia o respuesta a una condición inesperada, o bien en el código como resultado de la instrucción throw.
- Su propósito principal es permitir que un programa continúe ejecutándose sin interrupciones graves, incluso si se produce una excepción
- Ejemplo:

```
try {
    // Código que podría generar una excepción
} catch (TipoDeExcepcion1 e) {
    // Manejo de TipoDeExcepcion1
} catch (TipoDeExcepcion2 e) {
    // Manejo de TipoDeExcepcion2
} finally {
    // Código que siempre se ejecutará
}
```

2. Bucle: For

ATENCIÓN

for (int i=0; i<10; i++) ; → si pongo esto (;' al final) no voy a tener errores ni de ejecución ni de compilación pero se va a ejecutar la sentencia una sola vez

→ **For Avanzado:** utilizado para recorrer los objetos=

- más óptimo
- si o si recorre del principio al final y NO de manera inversa
- sintaxis:

```
for (objeto: colección)
{ sentencias; }
```

```
String[] nombres = {"Juan", "Pedro", "Dario",
"Carlos"};
```

```
for(String nombre:nombres)
System.out.println("Los nombres son: " + nombre);
```

→ **While/Do While**

• Ejemplo 1

```
1) while (condición)           // Primero se verifica la condición
    { sentencias; }           // Si es verdadera se ejecuta el código
```

• Ejemplo 2

```

2) do
    { sentencias; } // Primero se ejecuta el código
while (condición); // Y si es verdadera se ejecuta otro ciclo

```

break: detiene la ejecución del ciclo actual, continuando en la sentencia siguiente del ciclo.
continue: en lugar de detener la ejecución del ciclo, inicia otra vez con la siguiente iteración

Array o Vector: contiene una colección de elementos del mismo tipo que JAVA almacena y manipula usando un índice de tipo entero.

- Se consideran objetos con métodos asociados. ej: el método length
- Se dimensionan en tiempo de ejecución
- ¿Cómo se crean?

```

int duracion[];
duracion[]=new int[4];

```

→ en la creación del array, el array se llena con el valor por defecto del tipo de dato elegido

ATENCIÓN:

1. nombre_del_array.length → me devuelve el tamaño del array
2. NUNCA MENOR IGUAL EN LA CONDICIÓN DEL FOR

Arreglos de más de una dimensión... Matrices

```

int matriz[][];
matriz[][]=new int[4][4];

```

```

int a[][] = new int[10][3]; // matriz de enteros(10x3)
a.length;                  // Es 10 (la 1er dimensión)
a[0].length;                // igual a 3

```

- **CARACTERÍSTICAS:**
 - Los límites del array se comprueban en tiempo de ejecución para evitar desbordamientos.
 - No se pueden rellenar sin haber declarado el tamaño con el operador new.
 - Se puede combinar la declaración y la inserción de elementos en el vector, usando un inicializador de array.
 - Inicializador de array: indica el conjunto de elementos que participan del mismo, separados por coma y encerrados entre llaves.
 - No requiere el uso del operador new ni ser dimensionado previamente.

ATENCIÓN:

No se pueden crear arrays estáticos en tiempo de compilación: `int lista[50];` → NO

Bibliotecas de Clases

- java.lang: clases que se aplican al lenguaje mismo, entre las cuales se incluye la clases Object, String y System. También tiene clases para tipos primitivos (Integer, Character, Float, etc.) → esta no hace falta importarla
- java.util: clases de utilería, como Date y Random, y clases de colección de datos como Vector y nos va a permitir hacer el ingreso por teclado.
- java.io: clases para leer y escribir desde flujos (como las entradas y salidas estándares) y manejar archivos.
- java.net: clases para soporte de red, incluidos Socket y URL (una clase para representar referencias a documentos en WWW).
- java.awt: clases para implementar una interfaz gráfica del usuario, incluyendo clases para Window, Menu, Button, Font, CheckBox, etc. y también, las clases necesarias para el manejo de imágenes. → versión mejorada en la extensión en: javax.swing
- java.applet: clases para implementar applets JAVA.

→ Para utilizar en una aplicación alguna clase ya definida en JAVA, debe incluirse en la misma, el paquete de clases que la contiene.

→ ¿Cómo se ingresa una clase? → biblioteca.import.*

Entrada Estándar:

- esta clase NO forma parte de java.lang (librería estándar de Java)
- tenemos que importar la librería o una clase de la librería(mas optimo)
 - ◆ `import java.util.*;`
- tenemos que crear un objeto de la clase **Scanner**
- tipo de dato nombre de la variable = `stdin.nextTipodedato`
 - ◆ caso string: pongo solo `.next` o `.nextline`
- sentencia completa:

```
import java.util.*;
...
Scanner stdin = new Scanner(System.in);
int edad;
System.out.println("Ingrese la edad: ");
edad=stdin.nextInt();
System.out.println("Edad: " + edad);
```

Datos de Entrada y Salida

- *Stream*: conjunto de facilidades asociadas a los procedimientos de conversión de objetos en secuencias de caracteres y viceversa.
 - *stream* = flujo secuencial de datos.
- Archivos de entrada
 - Para leer archivo:
`FileInputStream InputFile=new FileInputStream ("en.data")`

- Archivo de salida
 - Para grabar datos en un archivo:


```
FileOutputStream outputFile=new FileOutputStream("s.data")
```
- Salida
 - estándar → objeto *out* de la clase *System* (pantalla (tb lo podemos cambiar a impresora, archivo, etc)
 - con formato → método *printf*

```
System.out.printf("Nombre: %s, Edad: %d%n", nombre, edad);
```
- Entrada:
 - estándar → teclado (lo podemos cambiar a un archivo por ejemplo)

Ingreso por teclado:

 - como ingreso:
 1. Creo el objeto:


```
Scanner stdin = new Scanner (System.in);
```
 2. Ejecuto algun metodo de lectura


```
double n = stdin.nextDouble();
```

 - `nextDouble()`
 - `nextInt()`
 - `nextLine()`
 - `next()`
 - todo ingreso por teclado tiene que estar validado. → ej. no puedo ingresar 13 en meses.
para chequear el tipo de dato:
 - `hasNextInt()`
 - `hasNextLong()`
 - `hasNextByte()`
 - `hasNext()`

ATENCIÓN: estoy obligada a limpiar el buffer cuando hago cambio del tipo de dato.
¿Cómo se hace ? → `stdin.nextLine();`

Funciones: pequeño programa en sí mismo que puede ser llamado desde el resto del programa.

Aclaración: En JAVA no existen funciones globales, porque cada función forma parte de la declaración de la clase con la que está relacionada.

Diferencia con C: Pueden ser definidas antes de o después de ser utilizadas

- **Tipos de acceso:**
 - **NUEVO!** → *package*: acceso solo para los objetos de la clase del paquete

¡ATENCIÓN! se pueden declarar **CON** o **SIN** el tipo de acceso.

- **Parámetros o argumentos:** valor específico que se utiliza en las tareas de la función
 - ¿Dónde deben estar ubicados?
 - Al definir una función se debe especificar el tipo de dato en cada uno de los parámetros utilizados (no existe la propagación con el uso de la coma).
 - alcance local exclusivamente.
 - , cuando un método llama a otro, los valores de los parámetros no están disponibles como tampoco las variables declaradas
 - no necesariamente todas las funciones aceptan parámetros o argumentos cuando son llamadas.
 - Distintos tipos de parámetros:
 - Parámetros Formales:** los nombres de las variables
 - representan una copia de los argumentos actuales
 - reservan una nueva porción de memoria
 - Argumentos actuales:** los valores reales
 - Los argumentos actuales pasan por referencia, cuando los formales NO representan una copia de los actuales.

Aclaración:

1. Solamente los objetos son pasados al cuerpo de una función por referencia. Así, cualquier modificación en sus variables afecta al objeto original.
2. No existe la posibilidad de utilizar argumentos por referencia con tipos primitivos.

Parámetros dinámicos:

- Se utilizan cuando se desconoce la cantidad de parámetros que se requiere
- Como? → “...” (3 puntos) entre el tipo de dato y el nombre del parámetro.

```
int calculo (int a, int b, int ... numero)
cálculo(4,5,67,9,2,4) //dos primeros
obligat cálculo(14,23,3,4)
```

→hay parametros dinamicos y estaticos: si uso los dos pongo primero los estáticos y después los dinámicos

- **Sobre carga de funciones (overloading):** cuando existen múltiples definiciones para una misma función.
 - definida en distintas clases
 - distintos tipos o cantidad de argumentos → IMP: no ambigüedad entre args.

- ejemplo:

```
public void mostrar (int x) {
    System.out.println ("Valor entero: " + x); }

public void mostrar (float x) {
    System.out.println ("Valor float: " + x); }

public void mostrar () {
    System.out.println ("Sin argumentos"); }

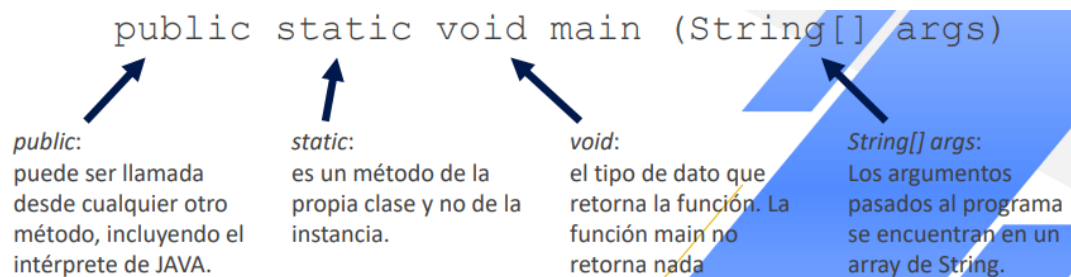
public int mostrar (int x)                // ERROR ! !
{ return x; }
```

ATENCIÓN:

- si pasa por parámetro → 12.5 → lo toma como double → error
→ 12.5f → le digo que lo trate como float
- si llamo: mostrar(25) y no existe una que reciba int → va a llamar a la que recibe float, va a tomar el 25 como un float

• Función Main:

- Para pasar argumentos a los programas, estos deben ser especificados en la línea de comandos al ejecutar la aplicación.
- Los argumentos informados se almacenan en un array de string, el cual pasa al método main() de la aplicación.
- args es el nombre del array de strings que contiene la lista de argumentos.
- A diferencia de C, JAVA no almacena en la primera posición del array (args[0]) el nombre del programa ejecutado, sino que guarda el primer argumento informado.
- Todos los argumentos son tratados como datos de tipo String.



Observación: En Java, cuando vos creás un array de enteros (int[]), todos sus elementos se inicializan automáticamente en 0.

Observación: El tipo de dato para cadenas de texto debe escribirse con la 'S' mayúscula: String, no string.

Definiciones hasta ahora:

Término	Definición
Coerción de Tipos de Datos	Transformar de modo explícito el tipo de dato de una variable en otro.
Prototipo	Declaración de método sin cuerpo.
Variable Local	Variable que debe ser inicializada.
Tipo de Datos	Conjunto específico de valores, junto con operaciones que pueden ser aplicadas a esos valores.
Parámetro Formal	Argumento que aparece en la cabecera del método.
Semántica	Conjunto de reglas que determina el significado de las instrucciones escritas en un lenguaje de programación.
Ámbito	Contexto de un identificador en un programa.
Literal	Valor que aparece en un programa.
Referencia	Tipo de parámetro que recibe la ubicación del argumento.
Sintaxis	Reglas formales mediante las que se rige la escritura de instrucciones válidas de un lenguaje de programación.
Expresión	Disposición de identificadores, literales y operadores que puede ser evaluada.
Condicional	Estructura de programación estructurada.
Conversión de Tipos de Datos por Extensión	Promoción implícita del tipo de dato.

Clases

- ¿Qué es una clase? → tipo de dato definido por el programador que contiene datos y funciones.
 - datos miembros: definen el estado.
 - funciones miembros: definen el comportamiento.

Inicialización de un objeto: debe ser declarado e instanciado (new)

Constructor: función especial, es llamada al instanciar un objeto

- Objetivo: crear e inicializar un objeto tal que contengan valores válidos.
- Si no se declara un constructor...

JAVA crea uno por default sin argumentos e inicializando con valores por defecto.

- Reserva memoria para cada uno de los datos miembros, de manera **secuencial** y **recursiva**
- No retorna valor. NO puede ser declarado estático.

Cómo creo una función constructora?

- La creo dentro de la clase y tiene el mismo nombre que la clase
- (Constructores Alternativos) puedo tener tantas funciones constructoras como distintos parámetros yo quiero que tenga. (SobreCarga)

Asignación de objetos

→ es otra forma de inicializar un objeto ya existente a base de un objeto ya declarado.

Eso significa que al modificar un atributo en un objeto se va a observar el cambio en el otro objeto.

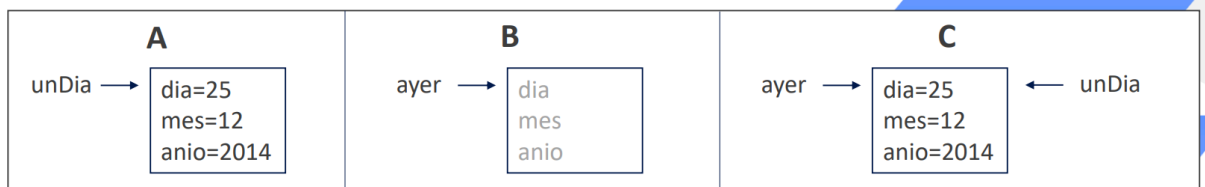
Ejemplo:

A) `Fecha unDia = new Fecha(25,12,2014);`

B) `Fecha ayer;`

C) `ayer = unDia;`

El mismo efecto se puede obtener declarando e inicializando en la misma línea: `Fecha ayer = unDia;`



ATENCIÓN: No se copia el contenido sino que ambas variables pasan a referenciar al mismo espacio de memoria.

Métodos de acceso

Atencion: variable.get() → valor
variable → dir memoria

Herencia: mecanismo para definir clases derivadas a partir de clases ya existentes

- La clase derivada hereda todas las propiedades y miembros de la clase existente, que recibe el nombre de clase base o clase padre o superclase.
 - Miembros public y protected → accede como si fuesen suyos
 - Miembros private → no tiene acceso
- Los miembros heredados por una clase derivada pueden, a su vez, ser heredados por más clases derivadas de ella.

ATENCIÓN

Si tengo una función x que está en mi objeto padre, entonces va a ejecutar la función del objeto padre por más que la llame desde el hijo.

Pero si tengo dos funciones x con los mismos parámetros una en el padre y otra en el hijo, entonces, y lo llamé desde el hijo entonces → va a llamar a la primera que encuentra, la del hijo.

- **Overriding:** es la forma de crear un método en la subclase que tenga la misma identificación de un método en una superclase.
 - oculta el método de la superclase.
- Los constructores NO son heredados...

Herencia: this y super

- **This:** hace referencia a miembros de la clase actual.
 - evita ambigüedad.
 - Se usa cuando un atributo se llama igual que una variable o parámetro.
- **Super :** hace referencia a métodos de la clase base
 1. método definido en una clase base, llamado desde una clase hija.

```
@Override
void hacerSonido() {
    super.hacerSonido();
    System.out.println("El perro ladra.");
}
```

2. para hacer referencia a algún constructor de la clase padre(super reemplaza al constructor). Ejemplo:

IMPORTANTE:

¿Cómo compilo? → cd scr ENTER(para entrar al directorio correcto) javac Nombre.java ENER
¿Cómo ejecuto? → java App ENTER

Destrucción de Objetos

→ Java NO utiliza destructores. Libera memoria de forma automática (Garbage Collector)

- ¿Cómo funciona? en tiempo de ejecución rastrea cada objeto creado y libera si es necesario
- Ventajas: • detecta objetos no utilizados • reutilización de la memoria • no necesita explícita liberación (destructores)
- Desventaja: • incrementa el tiempo de ejecución

Método de finalización: miembros especiales de la clase que se invocan antes de que el objeto sea recolectado como basura y reclame memoria.

→ ***finalize()***

- Cuando no se declara “crea” uno por default sin argumentos.
- Uso:
 - optimizar la remoción de un objeto
 - eliminar las referencias a otro objeto
 - liberar recursos externos

Cláusula *final*: Impide la redefinición de métodos, la extensión completa de clases y el cambio de valores de los atributos

Podemos hacer uso de esta clase en 3 “formas”:

- En Atributos: No puede cambiar su valor
 - ```
class Circulo {
 public final static float PI = 3.14;
}
```
- En Métodos: No puede ser reescrito (Override)
  - ```
class Empleado {  
    public final void Sueldo(int porc){ }
```
- En Clases: No puede tener hijos. Si lo hacemos tenemos errores en tiempo de compilación
 - ```
final class Ejecutivo{
 //definición de la clase
}
```

## **Clases y métodos abstractos**

Métodos abstracto:

- Un método abstracto es un método declarado en una clase para el cual esa clase no proporciona la implementación (no tiene código).
- Están disponibles en las clases derivadas, las cuales deben definir el mismo (polimorfismo).
- no tiene implementación → no tiene llaves y en cambio tiene la firma abstract (si ponemos llaves y abstract, error en tiempo de compilación)

### Clase abstracta:

- Una clase abstracta es aquella que tiene al menos un método abstracto.
- Una clase que extiende a una clase abstracta debe implementar los métodos abstractos, o bien, volverlos a declarar como abstractos, con lo que ella misma se convierte también en clase abstracta.
- Tiene que tener al menos un método abstracto
  - En el caso de que una clase hija no implemente el método abstracto necesario, la clase hija va a ser abstracta tb.
- Se pueden crear referencias a clases abstractas.  
`FiguraGeometrica figura;`  
`FiguraGeometrica figura = new Circulo();` → esto sí  
Pero una clase abstracta no se puede instanciar (no se pueden crear objetos)  
`FiguraGeometrica figura = new FiguraGeometrica();` // La línea de arriba va a dar error
- Esto es coherente dado que una clase abstracta no tiene completa su implementación y algo abstracto no puede materializarse.
- ¿Para qué sirve entonces? Para implementar el Polimorfismo de herencia, también llamado **Polimorfismo** Dinámico.
- Ejemplo:

```
Vehículo v = new Auto();
v.frenar();
if (v instanceof Auto) { → antes de castear tengo que asegurarme que el objeto es del tipo que quiero casteando
 ((Auto)v).encender(); → le estoy diciendo que el obj v sea tratada como un auto
}
```

### Miembros static:

- Cuando el valor es siempre el mismo, no lo voy a guardar para todos los objetos, lo guardo una sola vez.  
Ejemplo: todos mis autos son Toyota, no voy a guardar toyota para cada auto.
- existe aunque no existan objetos de esa clase

ATENCIÓN: no puedo referenciar métodos estáticos desde un objeto, es decir, no puedo cambiar el valor para un solo objeto, porque es “global”.

- Implementación: Lo ideal sería que cada clase vaya sumando +1 a su total de objetos DENTRO DE LA PROPIA CLASE a medida que se van creando → uso un método estático.  
Ejemplo:

```

public class MiClase {
 private static int contador = 0;
 public MiClase() {
 contador++; // Incrementa el contador cada vez que se
 // crea una instancia de la clase
 }
 public static int getContador() {
 return contador;
 }
}

System.out.println("Contador: " + MiClase.contador); // Imprime la
cantidad de objetos creados

```

## Array de Objetos

- **CONDICIÓN:** la clase debe tener un método constructor vacío. (evitar errores)
- No crece ni decrece su tamaño en tiempo de ejecución.
- Cada posición del vector funciona como un objeto independiente.
- Pasos:

1. Declaro y Dimensiono. Ejemplo:

```
Fecha vto[]=new Fecha[8];
```

2. Construir:

```

for (int i=0; i<8; i++){
 vto[i] = new Fechas();
}

```

- Puedo declarar y construir a la vez. Ejemplo (un array de 3 posiciones):

```

Fecha vto[] = { new Fecha(05,09,2023),
 new Fecha(03,10,2023),
 new Fecha(04,11,2023)
};

```

- Que NO puedo hacer: `System.out.print(v)` → es una direc de mem  
`System.out.print(vto)` → es una direc de mem  
`System.out.print(vto[i])` → es una direc de mem

## Recorridos del Array

Con ciclo for:

```

for(int i=0; i<vto.length;i++){
 System.out.println("Dia: " + vto[i].getDia());
 System.out.println("Mes: " + vto[i].getMes());
}

```

Con ciclo for avanzado:

```

for(Fecha v:vto){
 System.out.println("Dia: " + v.getDia());
}

```

```

 System.out.println("Mes: " + v.getMes());
 }

```

**Paquetes:** permite agrupar clases e interfaces

- nombre del package puede ser varios nombres unidos por puntos
- clases del mismo paquete → mismo directorio
- ¿Cómo hago para incluir una clase en un paquete? → la palabra reservada `package` y el nombre del paquete en la primera línea del programa: `package nombrePaquete;`
- si no tiene paquete? → por defecto en el mismo directorio que el archivo fuente.
- ¿Cómo uso clases de otros paquetes? → `import`
  - solo voy a poder usar las *públicas*

**Control de acceso o visibilidad**

| Visibilidad                       | Public | Protected | Private | Package |
|-----------------------------------|--------|-----------|---------|---------|
| Propia Clase                      | Si     | Si        | Si      | Si      |
| Otra Clase del Paquete            | Si     | Si        | No      | Si      |
| Otra Clase Fuera del Paquete      | Si     | No        | No      | No      |
| Clase Derivada dentro del Paquete | Si     | Si        | No      | Si      |
| Clase Derivada fuera del Paquete  | Si     | Si        | No      | No      |

**¡ATENCIÓN!** no puedo acceder desde el programa principal de manera directa a los atributos de un objeto → perdemos encapsulamiento!!!

**Objetos como miembros de una clase** → los objetos aislados no sirven, colaboran o intercambian información manteniendo relaciones

- Como se implementa? → agregando referencias a objetos de la clase destino como atributos de la clase origen

**Vamos a ver distintos tipos de relaciones...**

### 1. **Composición**

- clase principal compone un objeto nuevo de la clase secundaria
- el objeto miembro no tiene sentido si no existe la clase principal
- Es responsabilidad de la clase primaria construir el objeto de la clase secundaria.

```
public Persona(String pNombre, String pDir) {
 nombre = pNombre;
 direccion = pDir;
 fechaNac = new Fecha(); //no lo recibo como param.
}
```

Constructor vacío:

```
public Persona() {
 fechaNac = new Fecha();
}
```

**No puede recibir el objeto fecha como parámetro.**

- Que pasa si hago un get de el atributo compuesto por la clase secundaria?

```
System.out.println(pers1.getFechaNac()); → Fecha@36baf30c
```

Me devuelve la direccion de memmoria!!!

Como tengo que hacer?

```
System.out.println(pers1.getFechaNac().getDia()); → 25
```

## 1. Composición con cardinalidad multiple

Ejemplo: ahora queremos modelar los estudios que realizo una persona

**ATENCIÓN:** La clase secundaria nunca se recibe por parámetro, no existe, lo tengo que construir

¿Cómo se hace? —————

1. Agregamos un nuevo atributo en la clase paises:

```
public class Persona { // ... como atributo, agregamos:
 private EstudioRealizado estudios[] = new EstudioRealizado[10];
```

2. Recorrido: recorro el array y en cada una de sus posiciones construyo/compongo el objeto

```
public Persona() {
 fechaNac = new Fecha();
 for (int i=0; i<estudios.length; i++){
 estudios[i]=new EstudioRealizado();
 }
}
```

3. Constructor Alternativo: como hago para no reescribir el for en todos los constructor?

**ATENCIÓN:** para invocar a un constructor dentro de otro se usa → `this()`;

```
public Persona(String pNombre, String pDir, Pais pPais, Pais[]
pVisitados) {
 this(); //siempre primero
 nombre = pNombre;
 direccion = pDir;
```

```

nacionalidad = pPais;
paísesVisitados = pVisitados;
}

```

**Cuando tengo dos invoco a uno y para el otro hago la asignacion manual?**

4. SET: Para la posición necesito pasarle la posición que voy a setear.

```

public void setEstudios (int inicio, int fin, char tipo,
 String instituto, int pos)
{
 estudios[pos].setAnioInicio(inicio);
 estudios[pos].setAnioFin(fin);
 estudios[pos].setTipo(tipo);
 estudios[pos].setInstituto(instituto);
}

public EstudioRealizado[] getEstudioRealizado(){
 return estudios;
}

```

Métodos declarados  
en la clase  
EstudioRealizado

## 2. Agregacion

- el constructor de el objeto personas va a recibir el objeto/array país
- La existencia del objeto miembro es **independiente** a la del objeto principal.
- Antes de crear la clase Primaria tengo que haber creado el objeto de la clase secundaria.
  - clase principal recibe como parametro al objeto de la clase secundaria.
- como se hace la programación de la clase Principal:

```

public class Persona {
 private String nombre, direccion;
 private Fecha fechaNac;
 private Pais nacionalidad;
 public Persona(String pNombre, String pDir, Pais pPais) {
 nombre = pNombre;
 direccion = pDir;
 fechaNac = new Fecha();
 nacionalidad = pPais;
 }
 public Persona(){
 fechaNac = new Fecha();
 }
 public Pais getNacionalidad(){
 return nacionalidad;
 }
 public void setNacionalidad(Pais pPais) {
 nacionalidad = pPais;
 }
 Pais paises[] = {new Pais(1, "Argentina"),
 Pais(2, "Uruguay"),
 Pais(3, "Brasil")}
}

```

```

 }
 Persona pers2 = new Persona("Chris", "LIR", Países[2]);
 System.out.println(pers2.getNacionalidad());
 System.out.println(pers2.getNacionalidad().getNombre());
}

```

## 2. Agregación con cardinalidad múltiple → usamos array o arrayList

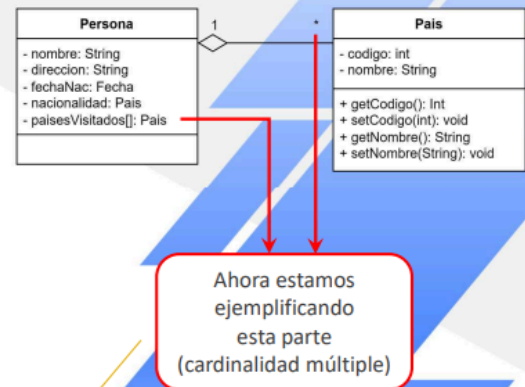
Que pasa cuando tengo... **CARDINALIDAD MÚLTIPLE**: usamos array o arrayList

```

public class Persona {
 private String nombre, direccion;
 private Fecha fechaNac;
 private Pais nacionalidad
 private Pais paisesVisitados[] = new
 Pais[5];

 public Persona(String pNombre, String pDir,
 Pais pPais,
 Pais[] pVisitados) {
 nombre = pNombre;
 direccion = pDir;
 fechaNac = new Fecha();
 nacionalidad = pPais;
 paisesVisitados = pVisitados;
 }
}

```



Pasos:

1. Armo el atributo array (HAY QUE DIMENSIONARLOS),  
`private Pais paisesVisitados[] = new Pais[5];`
2. Set y constructor: nada raro, recibo el array como parametro y lo asignó  
`paisesVisitados = pVisitados;`
3. **COMPLETAR**

## Sobrecarga del método String

Método: toString → devuelve una representación en string del objeto.

- Cuando alguien quiera mostrar el objeto, no me de la dirección de memoria, sino que me muestre un texto que defini yo.
- Muestra el nombre de la clase y el hash del objeto separado por @
- Todas las clases heredan este método por ser hijas de la clase Object.
- se lanza directamente cuando llamo a...`System.out.println()`.

**Atención:** si pongo el get solito estoy obligada a implementar toString, sino me va a devolver una dirección de memoria

## Clase Math: métodos estáticos

- parte de la librería → `java.lang.*`
- No podemos crear instancias/objetos de la clase
- es static (no necesita ser inicializada)
- public → puede ser llamada desde cualquier lugar.
- dos constantes. PI y E
- Métodos static:

```
Math.pow(2,3)=8
Math.sqrt(9)=3
Math.ceil(5.6)=6.0
Math.floor(5.6)=5.0
Math.round(5.6)=6
Math.round(5.4)=5
```

**Clase String:** opera sobre un objeto (El string es un objeto)

- librería → `java.lang.*`
- Métodos:
  - `equals()` → **NUNCA** == ('==', compara direcciones de memoria)
  - `equalsIgnoreCase()`
  - `indexOf()`
  - `length`
  - `etc...`

**Clase Random:** *generación* de objetos de manera aleatoria

- librería → `java.util.*`
- `Random r = new Random();`
- Métodos:
  - `r.nextInt()` : devuelve números aleatorios enteros
  - `r.nextInt(n)` : devuelve números aleatorios enteros positivos hasta n-1
  - `r.nextFloat()` : devuelve números aleatorios de tipo float
  - `r.nextDouble()` : devuelve números aleatorios de tipo double
  - `r.nextLong()` : devuelve números aleatorios de tipo long

**Clase Calendar:**

- librería → `java.util.*`
- es *abstracta*
- ¿Cómo lo creamos? → NO usamos new
  - `getInstance()`:  
`Calendar c = Calendar.getInstance() //devuelve fecha hoy`  
`c.getTime();`
- ¿Cómo hago si quiero la fecha de otro día?
  - `c.set` → cambiar la fecha
  - `c.add` → calcular una nueva fecha
- Existen constantes: (es un final) → me sirve para operar
  - `.moth`
  - `.year`
  - `.day`
  - `.minut`
  - `etc...`

## De aca para abajo no complete

**Interfases:** expresiones puras de diseño.

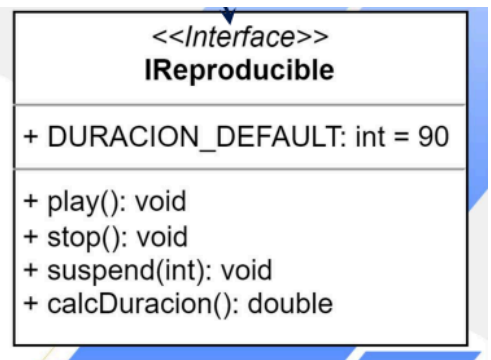
**DEFINICIÓN:** son conceptualizaciones no implementadas que sirven de guía para definir un determinado concepto y su comportamiento, pero sin desarrollar un mecanismo de solución.

Ventajas:

- Obligar a que ciertas clases utilicen los mismos métodos (nombres y parámetros).
  - Abstrae métodos a un nivel superior.
  - Establecer relaciones entre clases que no estén relacionadas
  - Implementar herencia múltiple.(java no soporta herencia múltiple).
  - Organizar la programación.
- 
- es un archivo.class
  - todas las interfaces son públicas por default → no hace falta poner el public
  - suele no llevar atributos → puede tener un atributo constante
  - lleva métodos **abstractos**
  - Ejemplo:

```
public interface IReproducible {
 int DURACION_DEFAULT = 90;

 public void play();
 public void stop();
 public void suspend (int m);
 public double calcDuracion();
}
```



Necesitamos definir los objetos que van a usar la interfaz:

→ ¿cómo?

en la definición de la clase agregamos... **implements NombreInterfaz**

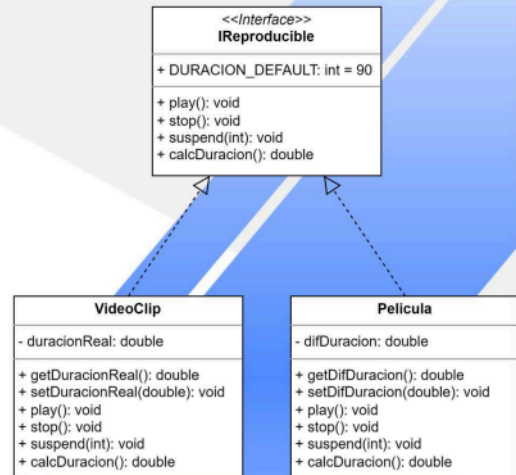
Ejemplo:

```

public class Peliculas implements IReproducible {
 private double difDuracion;

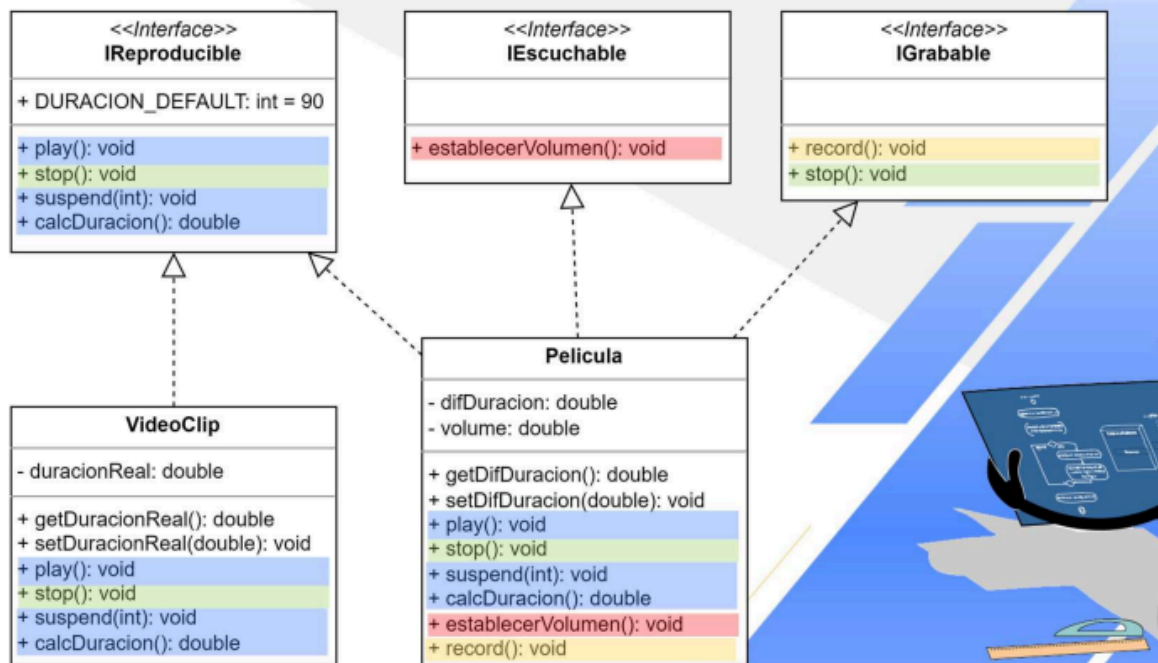
 public Peliculas(){}
 public Peliculas(double difDuracion){
 this.difDuracion = difDuracion;
 }
 public double getDifDuracion(){
 return difDuracion;
 }
 public void setDifDuracion(double difDuracion){
 this.difDuracion = difDuracion;
 }
 public void play(){
 System.out.println("Otro PLAY");
 }
 public void stop(){
 System.out.println("Otro STOP");
 }
 public void suspend(int min){}
 // definirlo aunque no haga nada
 public double calcDuracion(){
 return DURACION_DEFAULT + difDuracion;
 }
}

```



Remplazo al enum → **COMPLETAR**

Otro caso de uso de las interfaces... posibilidad de implementar **Herencia Múltiple**  
 → puedo tener mas de un padre



¿Cómo configuro esto? → uso coma ‘,’

```

public class Películas implements IReproducible, IGrabable, IEscuchable

```

Distintos tipos de interfaces (a partir de la versión 8):

- Como era antes de la 8... interfaz pura: TODOS los métodos abstractos

- Ahora... puedo implementar algún método, el comportamiento puede estar codificado en la interfaz base → cuando la implementación es común a todas.
- Distintos tipos:
  - Funcionales: 1 → abstracto, resto → abstracto
  - Virtuales: +1 → abstracto, +1 → implementado
- Si lo voy a codificar... tengo que agregar default

```
public interface ISaludoTarde {
 default void saludo(){
 System.out.println("Buenas tardes");
 }
}
```

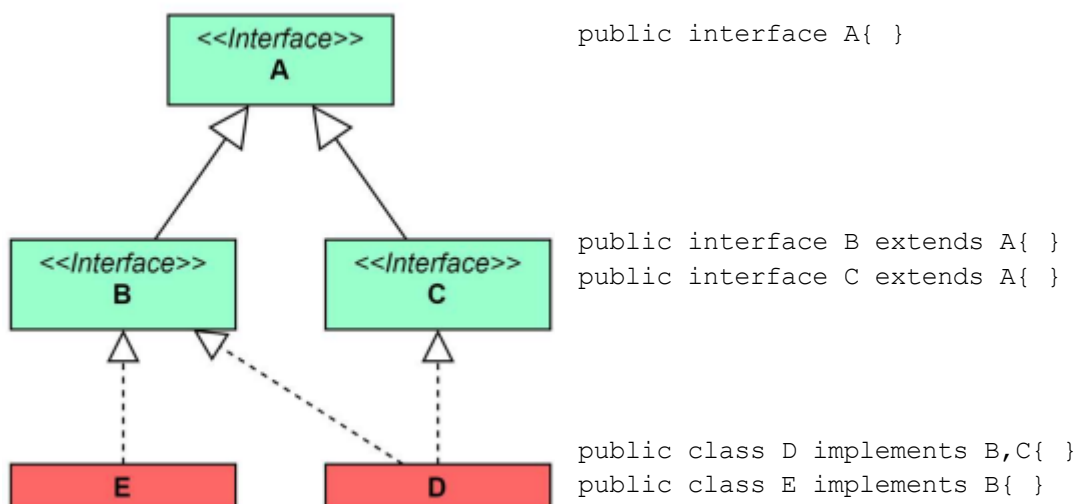
- Cuando tengo problemas con los nombres... **super**

```
public interface ISaludoTarde {
 default void saludo(){
 System.out.println("Buenas tardes");
 }
}
public interface ISaludoNoche {
 default void saludo(){
 System.out.println("Buenas noches");
 }
}
public class Saludos implements ISaludoTarde, ISaludoNoche {
 public void saludo(){
 ISaludoTarde.super.saludo(); } // para saber cuál invocar
}

// ISaludoTarde.saludo(): no es válido porque los métodos default
// no son estáticos
```

## Herencia de interfaces (sigue siendo permitida la herencia multiple)

¿Cómo se implementa? → **extends, implements**



## Interfaces vs Clases Abstractas

- Una clase abstracta puede incluir métodos implementados y no implementados o abstractos y miembros constantes y otros no constantes. Las interfaces contienen una

lista de métodos que pueden o no estar implementados e incluir la declaración de algunos miembros constantes.

- Una clase no puede heredar de dos clases abstractas, pero sí puede heredar de una clase abstracta e implementar una interface, o implementar dos o más interfaces. Las interfaces tienen una jerarquía propia, permitiéndose la herencia múltiple.
- Las interfaces proporcionan un mecanismo de encapsulación de los protocolos de los métodos sin forzar el uso de la herencia, asociando clases sin relación alguna.

**Atención:** los objetos tanto de una clase como una interfaz tiene acceso a... algo de castear **completar**

### **Repaso para el parcial**

ATENCIÓN: abreviatura del else if.

Opción 1:

```
int a=5;
if(a>5){
 Syso("la variable es mayor que 5");
}
else{
 Syso("la variable es menor que 5")
}
```

Opción 2:

```
Syso("la variable es" + (a>5)? "mayor" : "menor" + "que 5")
```